Compiladores, Linguagens Formais e Autômatos
203841

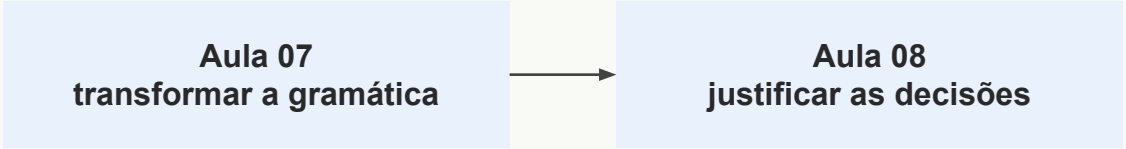
Aula 08: FIRST, FOLLOW e análise LL(1)

Como escolher uma produção usando o token atual?

Matheus Ferraroni Sanches

Objetivos e percurso

- Calcular anulabilidade, FIRST e FOLLOW.
- Construir uma tabela preditiva e localizar conflitos.
- Justificar decisões para entradas válidas e inválidas.



Ao fim da aula: justificar decisões com FIRST, FOLLOW e a tabela M.

Continuar, encerrar ou indicar erro?

```
print(2 + 3);

print(2);

print(2 3);
```

```
Expr' → PLUS Termo Expr'
      | MINUS Termo Expr'
      | ε
```

Depois do literal 2:

Token atual	Comportamento esperado
PLUS	A expressão continua.
RPAREN	A expressão pode terminar.
INT_LITERAL	Sequência não autorizada.

> Uma continuação opcional não autoriza ignorar um token inesperado.

Token atual e convenções

Token atual: o primeiro token ainda não consumido.

Objeto	Significado
A → α	Produção da gramática.
α ⇒ β	Uma etapa de derivação.
α ⇒* β	Zero ou mais passos.
ε	Cadeia vazia.
EOF	Marcador de fim de entrada.
INT_LITERAL	Um terminal sintático.

```
INT_LITERAL("2")
INT_LITERAL("3000")
```

Lexemas diferentes, mesmo tipo de terminal.

> Consultar não consome.
> ε não é token. EOF não é ε.

Anulabilidade

Uma construção pode desaparecer?

$A \rightarrow a \mid \varepsilon$

A pode gerar a e também pode gerar ε.

A é anulável ⇔ $A \Rightarrow^* \varepsilon$

$A \Rightarrow a$

$A \Rightarrow \varepsilon$

A é anulável se, e somente se, A consegue derivar a cadeia vazia em zero ou mais passos. Essa derivação pode ser direta ou passar por outros não terminais.

Anulável não significa “sempre desaparece”. Significa que existe uma derivação para ε.

Como encontrar os anuláveis?

```
Começar sem marcar nenhum não terminal.

Para cada produção A → α:
    marcar A quando α for vazio
    ou formado apenas por não terminais já marcados.

Repetir até uma passagem completa não mudar as marcações.
```

As informações acumulam. Uma passagem pode não bastar.

Exercício: anulabilidade indireta

Gramática auxiliar de anulabilidade | Símbolo inicial: S

```
S → A B c
A → a | ε
B → C
C → b | ε
```

- Determine os não terminais anuláveis.
- Justifique uma resposta que não use produção direta com ε.
- S pode gerar ε? Por quê?

A presença de ε em parte da regra não basta. Analise a sequência inteira.

Correção: o terminal obrigatório permanece

$A \Rightarrow \varepsilon$

$C \Rightarrow \varepsilon$

$B \Rightarrow C \Rightarrow \varepsilon$

Anuláveis: {A, B, C}

$S \Rightarrow A\ B\ c \Rightarrow B\ c \Rightarrow C\ c \Rightarrow c$

Toda derivação a partir de S mantém o terminal c obrigatório. Logo, S não é anulável.

A B é anulável. A B c não é anulável.

Exercício 1: sequências anuláveis

Classifique sequências, não apenas símbolos isolados

```
A → a | ε
B → b | ε
C → c
D → A B | C
```

Quais sequências podem gerar ε?

A B A C D

D B C A

Justifique cada resposta.

> Uma sequência é anulável quando todos os seus componentes podem desaparecer.

Correção do exercício 1: sequências

```
A B: sim
A C: não
D: sim
D B: sim
C A: não
```

A B ⇒* ε porque A ⇒ ε e B ⇒ ε.

`D ⇒ A B ⇒* ε; logo D B ⇒* ε.`

C sempre produz c.
Por isso A C e C A não são anuláveis.

Uma alternativa anulável basta para D; C impede a sequência que o contém.

Exercício 2: o efeito de C → ε

Símbolo inicial: S

```
S → A B
A → a | ε
B → b | C
C → c
```

1. Na gramática original, S é anulável? Por quê?

2. Acrescente a alternativa C → ε.

3. Quais não terminais são anuláveis agora?
Mostre uma derivação de S ⇒* ε.

Compare a gramática antes e depois da nova alternativa em C.

Correção do exercício 2: efeito de C → ε

Nova alternativa:
C → ε

Anuláveis depois:
{A, C, B, S}

Antes: S não era anulável; B mantinha b ou c.

Depois: C ⇒ ε e B ⇒ C ⇒ ε.

S ⇒ A B ⇒ B ⇒ C ⇒ ε.
No segundo passo, A desaparece porque A ⇒ ε.

A nova alternativa em C torna B e, então, S anuláveis.

First

FIRST: quais terminais podem iniciar?

FIRST(α) reúne os terminais que podem iniciar uma cadeia gerada por α.

```
Cmd → PRINT Arg | READ ID
Arg → ID | NUM
```

```
Cmd ⇒ PRINT Arg ⇒ PRINT ID
Cmd ⇒ PRINT Arg ⇒ PRINT NUM
Cmd ⇒ READ ID
```

```
Inícios: PRINT e READ

FIRST(Cmd) = {PRINT, READ}
```

> ε não é token. Ele entra em FIRST(α) somente se α puder gerar a cadeia vazia.

Exemplo completo: FIRST com ε

```
S → A B C | d
A → a | ε
B → b | ε
C → c | ε
```

Cadeias geradas

S ⇒* abc (início: a)
S ⇒* bc (início: b)
S ⇒* c (início: c)
S ⇒ d (início: d)
S ⇒* ε (cadeia vazia)

Sequência	FIRST
A B C	{a, b, c, ε}
S	{a, b, c, d, ε}

ε entra em FIRST(S) porque A, B e C podem desaparecer juntos.

Regra: percorra da esquerda para a direita

Para FIRST(X_1 X_2 … Xn):

1. Acrescente FIRST(X_1) sem ε.
2. Avance a X_2 só se X_1 puder desaparecer.
3. Pare no primeiro componente obrigatório.
4. Inclua ε só se todos puderem desaparecer.

Se B não puder desaparecer, até onde avançar?

A → a | ε
B → b
C → c | ε

FIRST(A B C) = {a, b}

O próximo símbolo só conta quando todo o prefixo anterior pode desaparecer.

Por que repetir o cálculo de FIRST?

```
S → A | d
A → B | c
B → b
```

Examinar S, depois A, depois B.

Momento	FIRST(S)	FIRST(A)	FIRST(B)
Inicial	∅	∅	∅
Passagem 1	{d}	{c}	{b}
Passagem 2	{c, d}	{b, c}	{b}
Passagem 3	{b, c, d}	{b, c}	{b}
Passagem 4	{b, c, d}	{b, c}	{b}

Pare após uma passagem completa sem mudanças. A ordem das regras altera só a velocidade.

Exercício 1 de FIRST: caso básico

Alternativas iniciadas por terminais

```
S → aA | bB
A → c
B → d
```

Calcule FIRST(S).

Observe o primeiro símbolo de cada alternativa de S.

Quais terminais podem aparecer no início de uma cadeia gerada por S?

Correção 1: início direto

```
S → aA: começa com a
S → bB: começa com b
```

FIRST(S) = {a, b}

Cada alternativa de S começa diretamente por um terminal.

A e B aparecem depois de a e b; seus terminais c e d não iniciam S.

FIRST(S) reúne os terminais que podem iniciar uma derivação de S.

Exercício 2 de FIRST: ε em A

Quando avançar para o próximo símbolo?

```
S → AB
A → a | ε
B → c
```

Calcule FIRST(S).

Encontre antes FIRST(A) e FIRST(B).

ε entra em FIRST(S)? Justifique.

> Se A puder desaparecer, examine B; confira se toda a sequência some.

Correção 2: A pode desaparecer

```
FIRST(A) = {a, ε}
FIRST(B) = {c}
```

FIRST(S) = {a, c}

A pode iniciar por a. Se A ⇒ ε, B fica na frente e traz c.

B não é anulável; S não gera a cadeia vazia, então ε fica fora.

Avance em AB porque A é anulável; pare em B porque B não é.

Exercício 3 de FIRST: dois ε

Ambos os componentes podem desaparecer

```
S → AB
A → a | ε
B → b | ε
```

Calcule FIRST(A), FIRST(B) e FIRST(S).

Quando ε deve entrar em FIRST(S)?

> Inclua ε somente se toda a sequência AB puder gerar vazio.

Correção 3: AB também pode sumir

```
FIRST(A) = {a, ε}
FIRST(B) = {b, ε}
```

FIRST(S) = {a, b, ε}

A traz a. Se A ⇒ ε, B passa à frente e traz b.

A ⇒ ε e B ⇒ ε; assim, S ⇒ AB ⇒* ε.

ε entra no FIRST da sequência quando todos os seus componentes somem.

Exercício 4 de FIRST: ε antes de b

Uma produção contém não terminal e terminal

```
S → Ab | c
A → d | ε
```

Calcule FIRST(A) e FIRST(S).

Ao escolher A → ε, qual símbolo fica no início de Ab?

ε entra em FIRST(S)?

Em S → Ab, o terminal b permanece mesmo quando A desaparece.

Correção 4: b fica na frente

```
FIRST(A) = {d, ε}
S → Ab: d ou b
S → c: c
```

FIRST(S) = {d, b, c}

`A → d traz d; A → ε deixa o terminal b no início.`

S → c acrescenta c. Como Ab sempre contém b, ε não entra.

A anulabilidade de A não torna a sequência Ab anulável.

Exercício 5 de FIRST: três símbolos

Percorra A, B e C em sequência

```
S → ABC
A → a | ε
B → b | ε
C → c | d
```

Calcule FIRST de S, A, B e C.

Até qual símbolo é preciso avançar em ABC?

S pode gerar ε?

> Avance apenas quando o símbolo anterior for anulável.

Correção 5: avance até C

```
FIRST(A) = {a, ε}
FIRST(B) = {b, ε}
FIRST(C) = {c, d}
```

FIRST(S) = {a, b, c, d}

A traz a; se A some, B traz b. Se ambos somem, C traz c ou d.

C não é anulável. Assim, ABC nunca some e ε fica fora.

Pare em C: ele inicia por c ou d e não pode desaparecer.

Exercício 6 de FIRST: recursão

Produções recursivas à direita

```
S → A | B
A → xA | y
B → zB | ε
```

Calcule FIRST(A), FIRST(B) e FIRST(S).

A recursão muda o primeiro terminal de xA ou zB?

> Observe o símbolo que aparece antes da chamada recursiva.

Correção 6: recursão após o terminal

```
FIRST(A) = {x, y}
FIRST(B) = {z, ε}
```

FIRST(S) = {x, y, z, ε}

A começa por x ou y. B começa por z ou produz vazio.

S → A | B une os conjuntos; ε vem da alternativa B → ε.

Em xA e zB, a recursão ocorre depois do primeiro terminal.

Exercício 7 de FIRST: expressões

```
E  → T E'
E' → + T E' | ε

T  → F T'
T' → * F T' | ε

F  → ( E ) | id
```

- Calcule FIRST(E), FIRST(E′), FIRST(T), FIRST(T′) e FIRST(F).
- Comece pelos casos diretos.

Nesta gramática, id, + e * são terminais; E′ e T′ são opcionais.

Correção 7: FIRST das expressões

Não terminal	Anulável?	FIRST
E	Não	{(, id}
E'	Sim	{+, ε}
T	Não	{(, id}
T'	Sim	{*, ε}
F	Não	{(, id}

FIRST(E) = {(, id}

F inicia por (ou id; como F não é anulável, T e E têm os mesmos inícios.

+ e * iniciam E′ e T′, mas não iniciam E ou T.

Exercício 8 de FIRST: ε indireto

B pode desaparecer dentro de A → Bd

```
S → AB | c
A → Bd | ε
B → b | ε
```

Calcule FIRST(B), FIRST(A) e FIRST(S).

Que terminal aparece quando B desaparece em A → Bd?

S pode gerar ε?

Em cada alternativa, avance quando o símbolo anterior puder desaparecer.

Correção 8: ε indireto em A

```
FIRST(B) = {b, ε}
FIRST(A) = {b, d, ε}
```

FIRST(S) = {b, d, c, ε}

Em A → Bd, B traz b ou desaparece e expõe d; A → ε acrescenta ε.

Em S → AB, A traz b e d; A e B podem sumir. S → c acrescenta c.

ε em FIRST(S) vem de S ⇒ AB ⇒* ε.

Follow

FOLLOW: o que pode vir depois?

FOLLOW(A) reúne os terminais que podem vir logo depois de A em uma derivação.

```
S → a A b
A → c | d

Depois de A vem b.
```

```
FOLLOW(A) = {b}

A → c ou A → d não muda o símbolo
seguinte.
```

> FIRST olha o início de A; FOLLOW olha o contexto depois de A.

Três regras para calcular FOLLOW

1. Símbolo inicial: sempre recebe `$` no FOLLOW.

Exemplo: Se S é o símbolo inicial, `$ ∈ FOLLOW(S)`.

2. Existe algo depois do não terminal: adicionamos o FIRST do que vem depois, sem `ε`.

Exemplo: `S → AB`, Adicionamos `FIRST(B) − {ε}` ao `FOLLOW(A)`.

3. Está no final da produção: recebe o FOLLOW do lado esquerdo. Isso também vale se tudo que vem depois puder gerar `ε`.

Exemplo: `S → aA`, Adicionamos `FOLLOW(S)` ao `FOLLOW(A)`.

Importante: O FOLLOW nunca contém `ε`.

Exemplo básico: um símbolo após A

```
S → AB
A → a
B → b
```

- Em S → AB, A é seguido por B: FIRST(B) = {b}.
- B está no fim e herda FOLLOW(S).

`FOLLOW(S) = {EOF} / FOLLOW(S) = {$}`

Conjunto	Resultado
FOLLOW(A)	{b}
FOLLOW(B)	{EOF}

Quando o sufixo pode desaparecer

```
S → AB
A → a | ε
B → b | ε

FIRST(B) = {b, ε}
```

B pode começar por b ou desaparecer.
Se B some, A herda FOLLOW(S).

Conjunto	Resultado
FOLLOW(A)	{b, EOF}
FOLLOW(B)	{EOF}

```
B → b: depois de A vem b.
B → ε: A fica no fim de S.
```

ε não entra no FOLLOW; ele permite herdar o contexto externo.

FOLLOW com várias produções

Produção	Ocorrência	Acrescente
S → Aa	A antes de a	a em FOLLOW(A)
S → B	B no fim	FOLLOW(S) em FOLLOW(B)
B → cA	A no fim	FOLLOW(B) em FOLLOW(A)
S inicial	fim da entrada	EOF em FOLLOW(S)
Resultado	FOLLOW(B)={EOF}	FOLLOW(A)={a,EOF}

A → b | ε não altera FOLLOW(A): procure A no lado direito.

Exercício 1 de FOLLOW: caso básico

Um terminal aparece imediatamente após A

```
S → aAb
A → c | ε
```

Calcule FOLLOW(S) e FOLLOW(A).

O que vem logo depois de A em S → aAb?

> Use todas as ocorrências do símbolo procurado.

Correção 1: terminal imediato

```
S é inicial → EOF
S → aAb: A precede b
```

FOLLOW(A) = {b}

```
A ocorrência de A é seguida diretamente pelo terminal b.
```

FOLLOW(S) = {EOF}. A produção A → c | ε não altera esse contexto.

FOLLOW descreve o que vem depois de A, não o que A gera.

Exercício 2 de FOLLOW: B anulável

Quando o símbolo seguinte pode desaparecer

```
S → AB
A → a | ε
B → b | ε
```

Calcule FOLLOW(S), FOLLOW(A) e FOLLOW(B).

O que pode vir após A quando B → ε?

> Se B sumir, examine o que pode vir após S.

Correção 2: B pode sumir

```
FIRST(B) = {b, ε}
FOLLOW(S) = {EOF}
FOLLOW(B) = {EOF}
```

FOLLOW(A) = {b, EOF}

`Em S → AB, B pode iniciar por b; inclua b em FOLLOW(A).`

Como B ⇒ ε, A também herda FOLLOW(S). B está no fim e herda FOLLOW(S).

FOLLOW nunca contém ε; a anulabilidade transfere o contexto.

Exercício 3 de FOLLOW: duas posições de A

Procure o mesmo símbolo em todas as produções

```
S → Ab | B
A → a | ε
B → cA
```

Calcule FOLLOW(S), FOLLOW(A) e FOLLOW(B).

Encontre as duas ocorrências de A.

> Cada ocorrência pode contribuir para o conjunto final.

Correção 3: reúna as ocorrências

```
S inicial → {EOF}
S → B: B herda EOF
```

FOLLOW(A) = {b, EOF}

```
S → Ab acrescenta b a FOLLOW(A).
```

B → cA deixa A no fim: ele herda FOLLOW(B) = {EOF}.

Uma única ocorrência não basta para calcular FOLLOW(A).

Exercício 4 de FOLLOW: sufixo BC

Dois símbolos anuláveis após A

```
S → ABC
A → a | ε
B → b | ε
C → c | ε
```

Calcule FOLLOW(S), FOLLOW(A), FOLLOW(B) e FOLLOW(C).

O que acontece se B e C desaparecerem?

Para A, examine a sequência inteira BC.

Correção 4: percorra o sufixo

```
FOLLOW(S) = {EOF}
FOLLOW(B) = {c, EOF}
FOLLOW(C) = {EOF}
```

FOLLOW(A) = {b, c, EOF}

`B traz b. Se B some, C traz c.`

Se B e C somem, A herda EOF. B herda EOF quando C some.

A recebe FIRST(BC) sem ε e, se BC some, FOLLOW(S).

Exercício 5 de FOLLOW: recursão

Três ocorrências de B em contextos distintos

```
S → AB | x
A → Bd | ε
B → bB | ε
```

Calcule FOLLOW(S), FOLLOW(A) e FOLLOW(B).

Localize B em S → AB, A → Bd e B → bB.

> Uma regra recursiva pode apenas repetir o que já sabemos.

Correção 5: recursão sem novidade

```
FOLLOW(S) = {EOF}
FOLLOW(A) = {b, EOF}
```

FOLLOW(B) = {d, EOF}

`S → AB dá EOF a B; A → Bd dá d a B.`

B → bB só copia FOLLOW(B) para ele mesmo; nada novo surge.

Analise todas as ocorrências e repita até estabilizar.

Exercício: FOLLOW das expressões

```
E  → T E'
E' → + T E' | ε

T  → F T'
T' → × F T' | ε

F  → ( E ) | num
```

- Calcule FOLLOW(E), FOLLOW(E′), FOLLOW(T), FOLLOW(T′) e FOLLOW(F).
- Comece por E inicial e marque cada ocorrência à direita.

Mesma estrutura do exercício de FIRST: num corresponde a id; × corresponde a *.

Correção: FOLLOW das expressões

Não terminal	Origem principal	FOLLOW
E	Inicial; (E)	{), EOF}
E'	Fim de E	{), EOF}
T	E′ anulável	{+,), EOF}
T'	Fim de T	{+,), EOF}
F	T′ anulável	{×, +,), EOF}

`FOLLOW(E) = {), EOF}`

E′ e T′ herdam o contexto. T recebe +; F recebe ×. Sufixos anuláveis passam o FOLLOW externo.

Se E′ ou T′ some, o contexto de fora atravessa o sufixo.

FIRST × FOLLOW: resumo das expressões

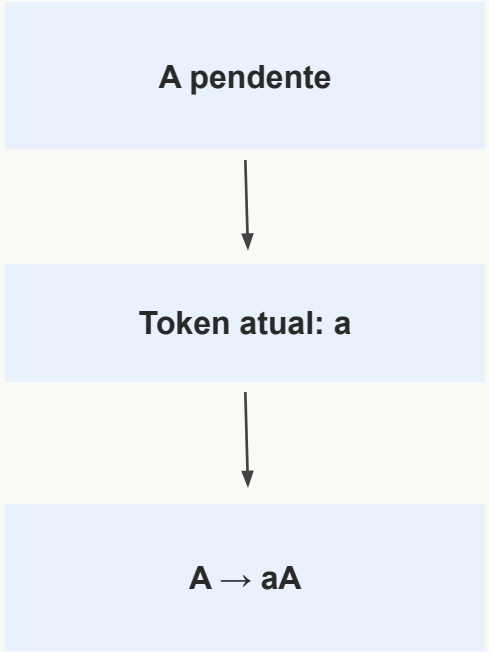
Não terminal	FIRST	FOLLOW
E	{num, (}	{), EOF}
E'	{+, ε}	{), EOF}
T	{num, (}	{+,), EOF}
T'	{×, ε}	{+,), EOF}
F	{num, (}	{×, +,), EOF}

FIRST pode conter ε; FOLLOW nunca. EOF marca o fim da entrada.

Análise preditiva LL(1)

O significado de LL(1)

Parte	Significado
Primeiro L	Leitura da entrada da esquerda para a direita.
Segundo L	Construção de uma derivação mais à esquerda.
1	Um token de consulta antecipada.



M[A,a] escolhe A → aA; consultar a tabela ainda não consome a.

A gramática: FIRST e FOLLOW

```
S → A B
A → a A | ε
B → b | c

Não terminais: S, A, B
Terminais: a, b, c
Fim da entrada: EOF
```

```
FIRST(S) = {a, b, c}
FIRST(A) = {a, ε}
FIRST(B) = {b, c}

FOLLOW(S) = {EOF}
FOLLOW(A) = {b, c}
FOLLOW(B) = {EOF}

B é obrigatório em S → AB; A pode
desaparecer.
```

Como preencher as células de M

1. LINHA
Para cada produção, use o não terminal à esquerda como linha.

2. COLUNAS PELO INÍCIO
Veja quais terminais podem aparecer primeiro no lado direito. Se um símbolo puder desaparecer, examine o próximo. Escreva a produção completa em cada coluna encontrada.

3. COLUNAS PELO VAZIO
Se todo o lado direito puder desaparecer, use as colunas de FOLLOW do não terminal da linha. Inclua EOF apenas se ele estiver nesse FOLLOW.

Em cada célula:

Escreva a produção inteira.

A mesma produção pode ocupar mais de uma coluna.

ε não é coluna. Na tabela completa, célula vazia indica erro; duas produções na mesma célula indicam conflito.

Tabela M em construção · 1/6

Terminal direto: S → aA começa por a.

Gramática	FIRST e FOLLOW
S → aA | b A → c	FIRST(S)={a,b} FIRST(A)={c}
Terminais: a, b, c Fim: EOF	FOLLOW(S)={EOF} FOLLOW(A)={EOF}

M	a	b	c	EOF
S	S → aA			
A				

Como decidimos

1. Linha S: examine S → aA.
2. FIRST(aA) = {a}; a inicia a produção.
3. Sem ε, use M[S,a]. FOLLOW não entra.

Tabela M em construção · 2/6

Não terminal sem ε: S → Ac | b.

Gramática	FIRST e FOLLOW
S → Ac | b A → a	FIRST(S)={a,b} FIRST(A)={a}
Terminais: a, b, c Fim: EOF	FOLLOW(S)={EOF} FOLLOW(A)={c}

M	a	b	c	EOF
S	S → Ac			
A				

Como decidimos

1. Linha S: examine S → Ac.
2. FIRST(A) = {a}; A não gera ε.
3. Pare em A: c não conta. Use M[S,a].

Primeiro símbolo anulável: S → AB | c.

Gramática	FIRST e FOLLOW
S → AB | c A → a | ε B → b	FIRST(S)={a,b,c} FIRST(A)={a,ε} FIRST(B)={b}
Terminais: a, b, c Fim: EOF	FOLLOW(S)={EOF} FOLLOW(A)={b} FOLLOW(B)={EOF}

Como decidimos

1. Linha S: examine S → AB.
2. ε ∈ FIRST(A), então avance para B.
3. FIRST(B)={b}, sem ε: M[S,b].

M	a	b	c	EOF
S		S → AB		
A				
B				

Produção vazia: o que pode vir depois de A?

Gramática	FIRST e FOLLOW
S → (A) A → id | ε	FIRST(S)={(} FIRST(A)={id,ε}
Terminais: (, id,) Fim: EOF	FOLLOW(S)={EOF} FOLLOW(A)={) }

M	(	id	)	EOF
S				
A			A → ε	

Como decidimos

1. Linha A: FIRST(ε) = {ε} para A → ε.
2. ε não é coluna; veja FOLLOW(A).
3. Em S → (A), vem): use M[A,)].

Produção vazia: A pode terminar a entrada.

Gramática	FIRST e FOLLOW
S → aA | c A → b | ε	FIRST(S)={a,c} FIRST(A)={b,ε}
Terminais: a, b, c Fim: EOF	FOLLOW(S)={EOF} FOLLOW(A)={EOF}

M	a	b	c	EOF
S				
A				A → ε

Como decidimos

1. A → ε: FIRST(ε)={ε}, sem coluna ε.
2. A termina S → aA: FOLLOW(A)={EOF}.
3. Preencha M[A,EOF] com A → ε.

Toda a sequência AB pode desaparecer.

Gramática	FIRST e FOLLOW
S → AB | c A → a | ε B → b | ε	FIRST(S)={a,b,c,ε} FIRST(A)={a,ε} FIRST(B)={b,ε}
Terminais: a, b, c Fim: EOF	FOLLOW(S)={EOF} FOLLOW(A)={b,EOF} FOLLOW(B)={EOF}

Como decidimos

1. Linha S: examine S → AB.
2. A e B podem sumir: ε ∈ FIRST(AB).
3. FOLLOW(S) = {EOF}: use M[S,EOF].

M	a	b	c	EOF
S				S → AB
A				
B				

Tabela M: preparação

Linhas S, A, B; colunas a, b, c e EOF. ε não é coluna.

Gramática	FIRST e FOLLOW
S → AB A → aA| ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Regra	FIRST(α): coluna ε: usar FOLLOW(X)

M	a	b	c	EOF
S	–	–	–	–
A	–	–	–	–
B	–	–	–	–

Regra: FIRST(α) indica a coluna; se α gera ε, use FOLLOW.

Tabela M · 1/8: M[S,a]

S → AB: a pode iniciar AB.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[S,a] ← S → AB a ∈ FIRST(AB)

M	a	b	c	EOF
S	S → AB	–	–	–
A	–	–	–	–
B	–	–	–	–

a ∈ FIRST(A), logo a ∈ FIRST(AB).

Tabela M · 2/8: M[S,b]

S → AB: A pode desaparecer e revelar B.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[S,b] ← S → AB b ∈ FIRST(AB)

M	a	b	c	EOF
S	S → AB	S → AB	–	–
A	–	–	–	–
B	–	–	–	–

b ∈ FIRST(B), logo b ∈ FIRST(AB).

Tabela M · 3/8: M[S,c]

S → AB: A pode desaparecer e revelar B.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[S,c] ← S → AB c ∈ FIRST(AB)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	–	–	–	–
B	–	–	–	–

c ∈ FIRST(B), logo c ∈ FIRST(AB).

Tabela M · 4/8: M[A,a]

A → aA: o lado direito começa pelo terminal a.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[A,a] ← A → aA a ∈ FIRST(aA)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	A → aA	–	–	–
B	–	–	–	–

FIRST(aA) = {a}; só M[A,a] muda.

Tabela M · 5/8: M[A,b]

A → ε: use o que pode vir depois de A.

Gramática	FIRST e FOLLOW
S → AB A → aA| ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[A,b] ← A → ε b ∈ FOLLOW(A)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	A → aA	A → ε	–	–
B	–	–	–	–

b ∈ FOLLOW(A); ε não é coluna.

Tabela M · 6/8: M[A,c]

A → ε: aplique a segunda coluna de FOLLOW(A).

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[A,c] ← A → ε c ∈ FOLLOW(A)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	A → aA	A → ε	A → ε	–
B	–	–	–	–

c ∈ FOLLOW(A); só M[A,c] muda.

Tabela M · 7/8: M[B,b]

B → b: o lado direito começa pelo terminal b.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[B,b] ← B → b b ∈ FIRST(b)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	A → aA	A → ε	A → ε	–
B	–	B → b	–	–

FIRST(b) = {b}; só M[B,b] muda.

B → c: o lado direito começa pelo terminal c.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Célula atual	M[B,c] ← B → c c ∈ FIRST(c)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	A → aA	A → ε	A → ε	–
B	–	B → b	B → c	–

FIRST(c) = {c}; só M[B,c] muda.

Tabela M completa: verificação

Oito inserções feitas, uma célula por vez.

Gramática	FIRST e FOLLOW
S → AB A → aA | ε B → b | c	FIRST(S) = {a,b,c} FIRST(A) = {a,ε} FIRST(B) = {b,c}
Terminais: a,b,c Fim: EOF	FOLLOW(S) = {EOF} FOLLOW(A) = {b,c} FOLLOW(B) = {EOF}
Conclusão	Gramática LL(1)

M	a	b	c	EOF
S	S → AB	S → AB	S → AB	–
A	A → aA	A → ε	A → ε	–
B	–	B → b	B → c	–

Sem conflitos: uma produção por célula;

Exercício 1 de M (fácil): escolha direta

Produções iniciadas por terminais.

```
S → aA | b
A → c | d
```

Calcule FIRST e FOLLOW de S e A.

Construa a tabela M.

> Preencha as células antes de ver a correção.

Correção 1: escolha direta

FIRST e FOLLOW indicam as colunas; nenhuma célula tem conflito.

NT	FIRST	FOLLOW
S	{a,b}	{EOF}
A	{c, d}	{EOF}

M	a	b	c	d	EOF
S	S→aA	S→b	–	–	–
A	–	–	A→c	A→d	–

Exercício 2 de M (fácil): uma alternativa ε

A pode desaparecer; B sempre produz b.

```
S → AB
A → a | ε
B → b
```

Calcule FIRST e FOLLOW de S, A e B.

Construa a tabela M.

Correção 2: ε usa FOLLOW(A)

A → ε entra na coluna b, pois FOLLOW(A) = {b}.

NT	FIRST	FOLLOW
S	{a,b}	{EOF}
A	{a, ε}	{b}
B	{b}	{EOF}

M	a	b	EOF
S	S→AB	S→AB	–
A	A→a	A→ε	–
B	–	B→b	–

Exercício 3 de M (intermediário): dois ε

A e B podem desaparecer antes do c.

```
S → ABc | d
A → a | ε
B → b | ε
```

Calcule FIRST e FOLLOW de S, A e B.

Construa a tabela M completa.

> Em FIRST(ABc), avance quando o símbolo anterior puder sumir.

Correção 3: ABc alcança c

A e B são anuláveis; c ainda é terminal obrigatório.

NT	FIRST	FOLLOW
S	{a,b,c,d}	{EOF}
A	{a, ε}	{b, c}
B	{b, ε}	{c}

M	a	b	c	d	EOF
S	S→ABc	S→ABc	S→ABc	S→d	–
A	A→a	A→ε	A→ε	–	–
B	–	B→b	B→ε	–	–

Exercício 4 de M: expressões

Use a gramática de expressões já trabalhada na aula.

```
E  → TE'
E' → +TE' | ε
T  → FT'
T' → *FT' | ε
F  → (E) | id
```

Calcule FIRST e FOLLOW de E, E', T, T' e F.

Monte a tabela M completa.

Colunas: id, +, *, (,) e EOF.
Coluna EOF: preencha apenas se a produção puder gerar ε e EOF estiver no FOLLOW do não terminal da linha.

Correção 4: tabela M das expressões

As produções com ε usam FOLLOW(E') e FOLLOW(T').

NT	FIRST	FOLLOW
E	{(,id}	{),EOF}
E'	{+,ε}	{),EOF}
T	{(,id}	{+,),EOF}
T'	{*,ε}	{+,),EOF}
F	{(,id}	{*,+,),EOF}

M	id	+	*	(	)	EOF
E	E→TE'	–	–	E→TE'	–	–
E'	–	E'→+TE'	–	–	E'→ε	E'→ε
T	T→FT'	–	–	T→FT'	–	–
T'	–	T'→ε	T'→*FT'	–	T'→ε	T'→ε
F	F→id	–	–	F→(E)	–	–

Exercício 5 de M: decisão LL(1)

Há anulibilidade e alternativas na mesma linha.

```
S → AC | b
A → Bd | ε
B → b | ε
C → c | ε
```

Calcule FIRST e FOLLOW de S, A, B e C.

Construa a tabela M completa.

A gramática é LL(1)? Justifique a célula decisiva.

> Compare as alternativas de S antes de concluir.

Correção 5: conflito em M[S,b]

S → AC e S → b são escolhas para o mesmo token b.

NT	FIRST	FOLLOW
S	{b,c,d,ε}	{EOF}
A	{b,d,ε}	{c,EOF}
B	{b,ε}	{d}
C	{c,ε}	{EOF}

M	b	c	d	EOF
S	S→AC S→b	S→AC	S→AC	S→AC
A	A→Bd	A→ε	A→Bd	A→ε
B	B→b	–	B→ε	–
C	–	C→c	–	C→ε

Não é LL(1): M[S,b] contém duas produções.

Utilização Tabela M

Tabela M das expressões

Gramática: E→TE' E'→+TE'|ε T→FT' T'→*FT'|ε F→id|(E)

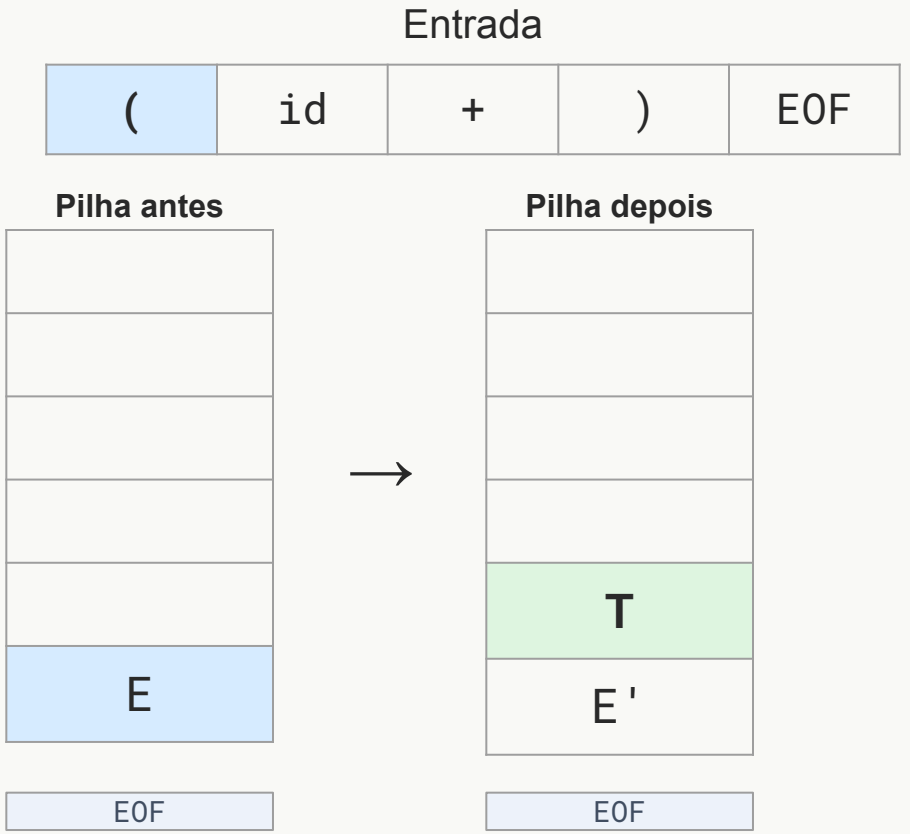
Entrada da simulação: (id +) EOF · Topo da pilha à esquerda

M	id	+	*	(	)	EOF
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

Uso da tabela M · passo 1/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

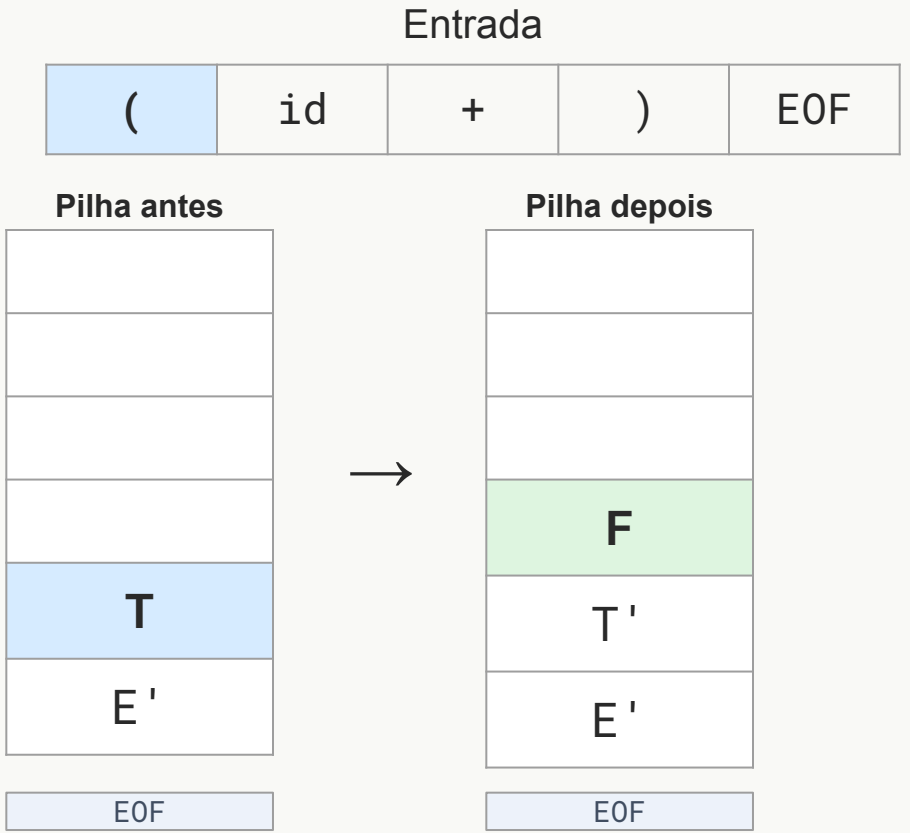


Ação: E → TE' (M[E,(])

Uso da tabela M · passo 2/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

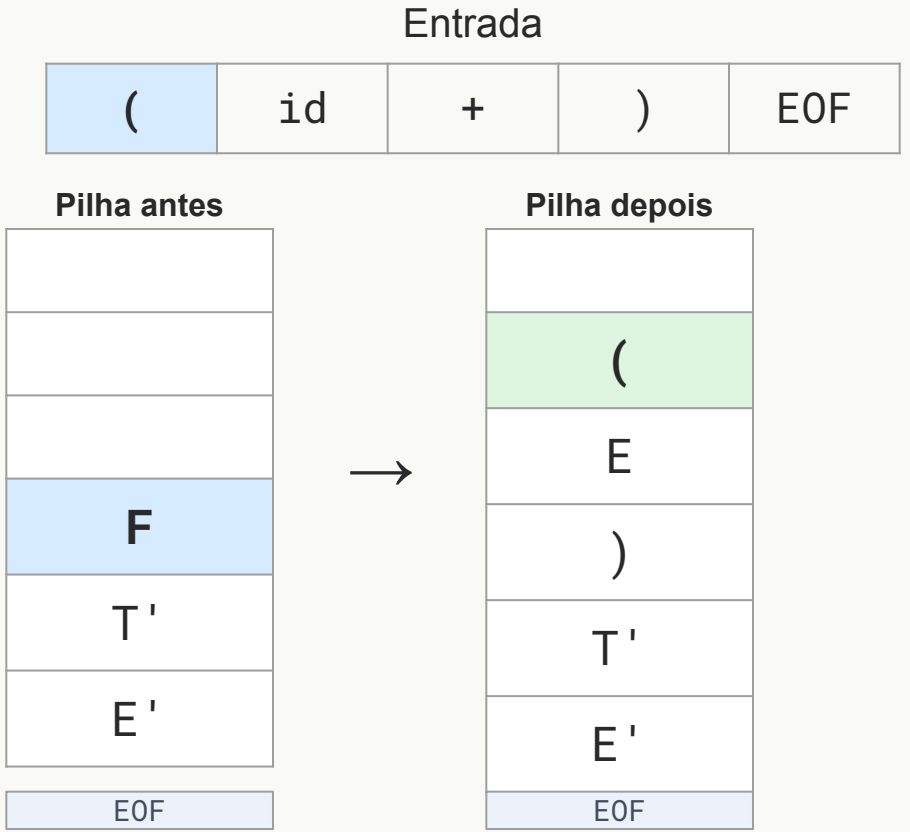


Ação: T → FT' (M[T,(])

Uso da tabela M · passo 3/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

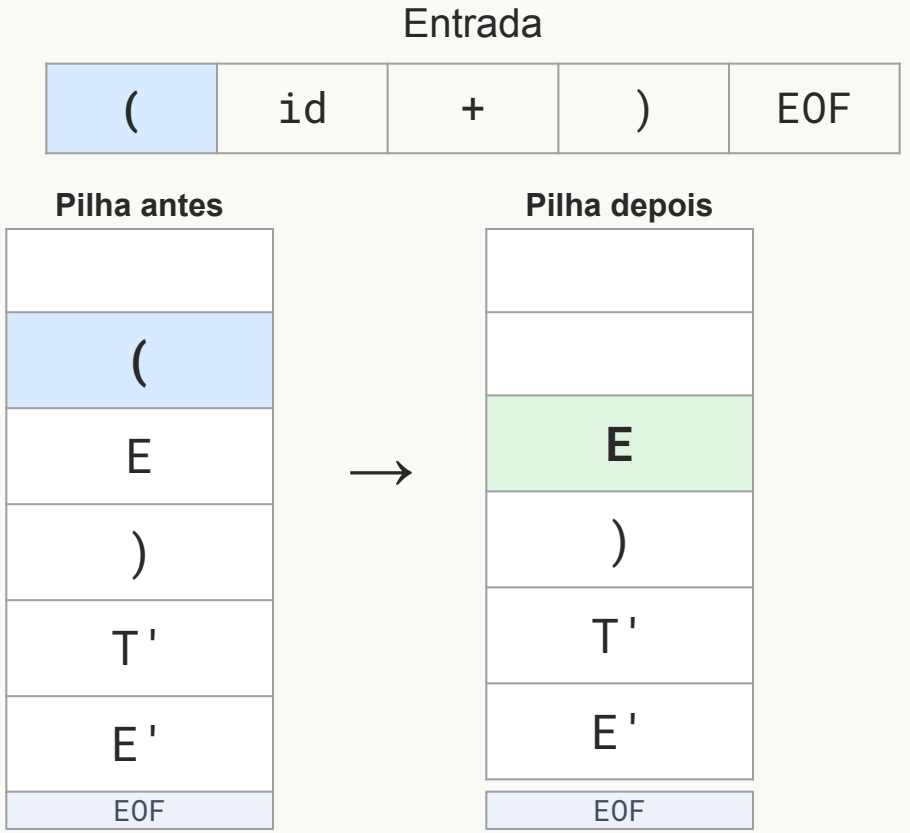


Ação: F → (E) (M[F,(])

Uso da tabela M · passo 4/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

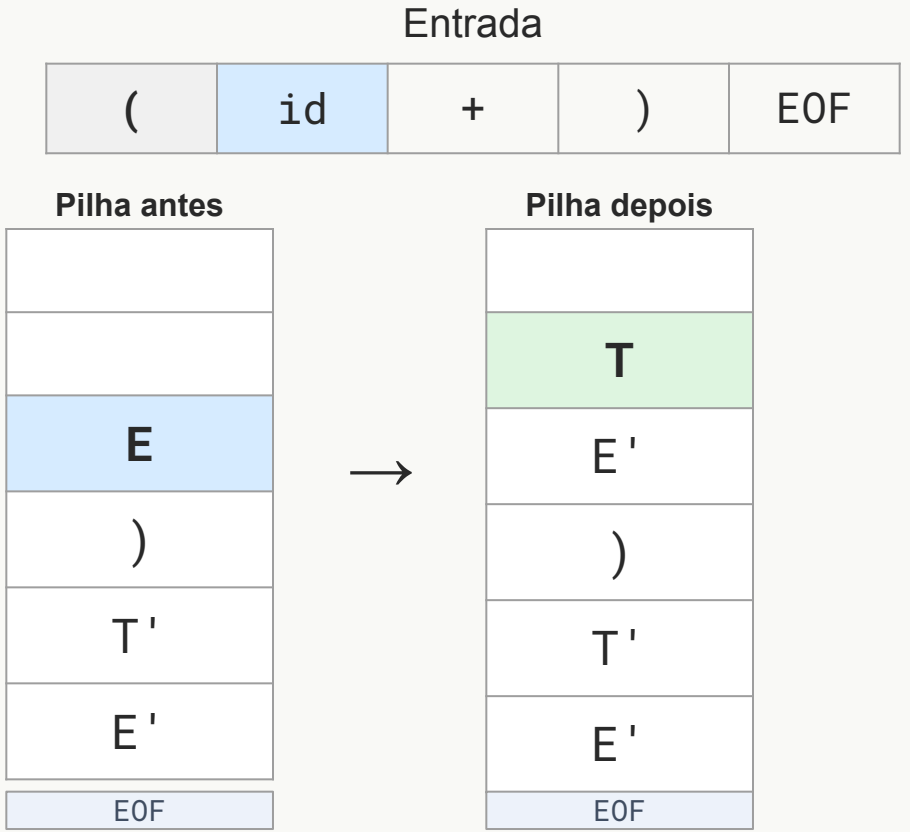


Ação: consumir (

Uso da tabela M · passo 5/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		



Ação: E → TE' (M[E,id])

Uso da tabela M · passo 6/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

Entrada

(	id	+	)	EOF

Pilha antes

T
E'
)
T'
E'
EOF

→

Pilha depois

F
T'
E'
)
T'
E'
EOF

Ação: T → FT' (M[T,id])

Uso da tabela M · passo 7/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

Entrada

(	id	+	)	EOF

Pilha antes

F
T'
E'
)
T'
E'
EOF

→

Pilha depois

id
T'
E'
)
T'
E'
EOF

Ação: F → id (M[F,id])

Uso da tabela M · passo 8/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

Entrada

(	id	+	)	EOF

Pilha antes

id
T'
E'
)
T'
E'
EOF

→

Pilha depois

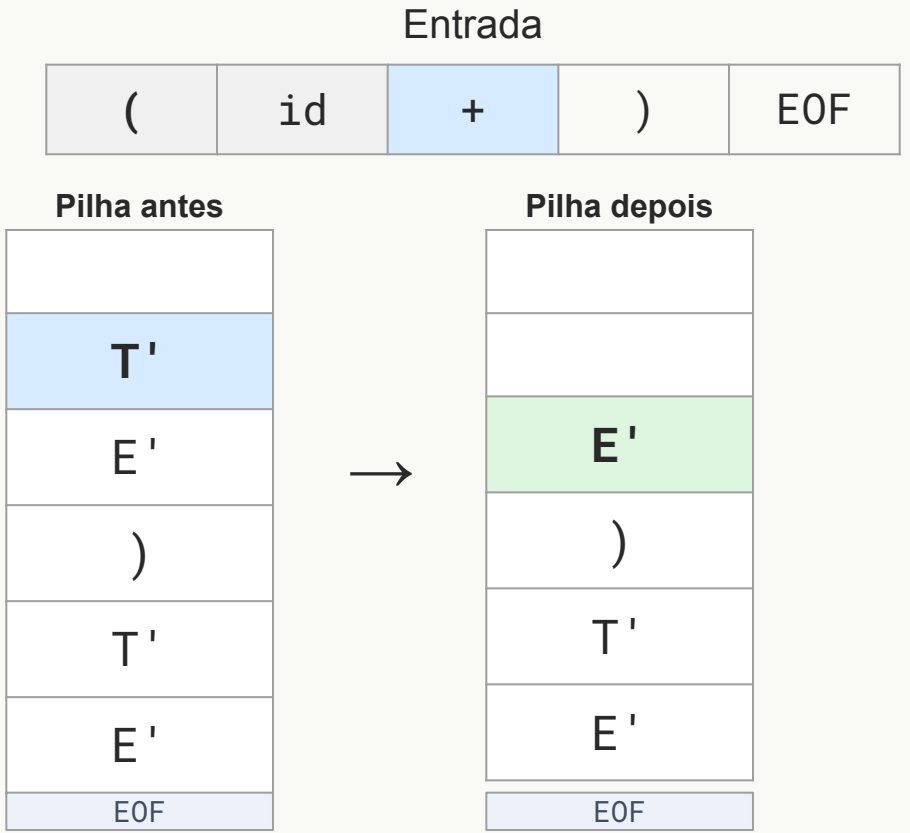
T'
E'
)
T'
E'
EOF

Ação: consumir id

Uso da tabela M · passo 9/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

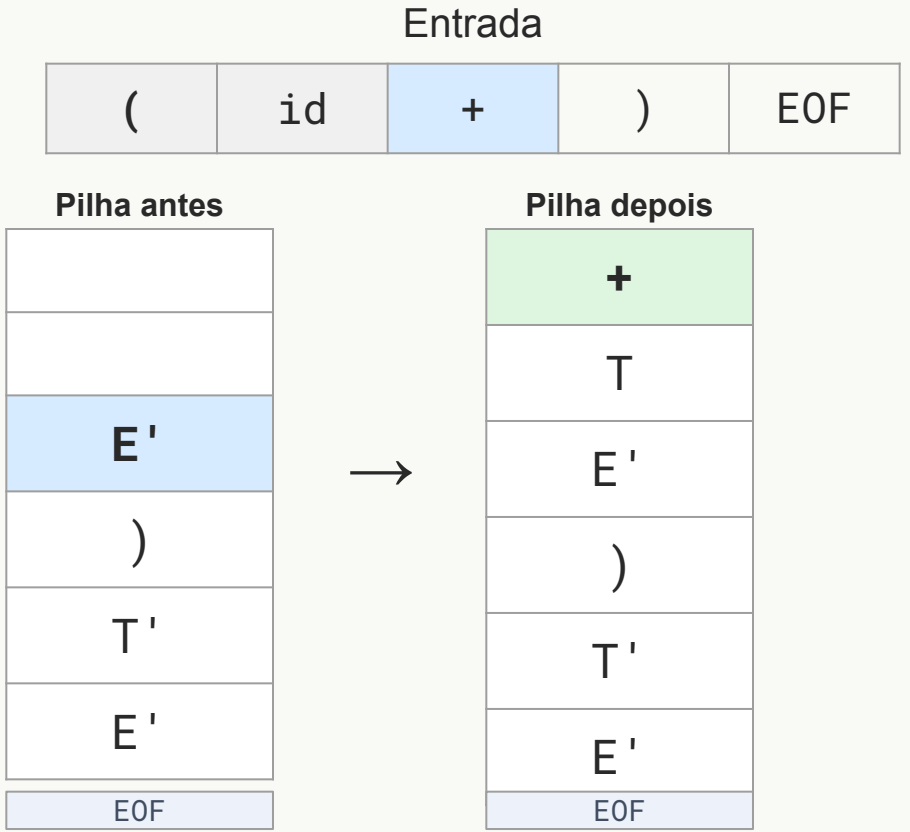


Ação: T' → ε (M[T',+])

Uso da tabela M · passo 10/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		



Ação: E' → +TE' (M[E',+])

Uso da tabela M · passo 11/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

Entrada

(	id	+	)	EOF

Pilha antes

+
T
E'
)
T'
E'
EOF

→

Pilha depois

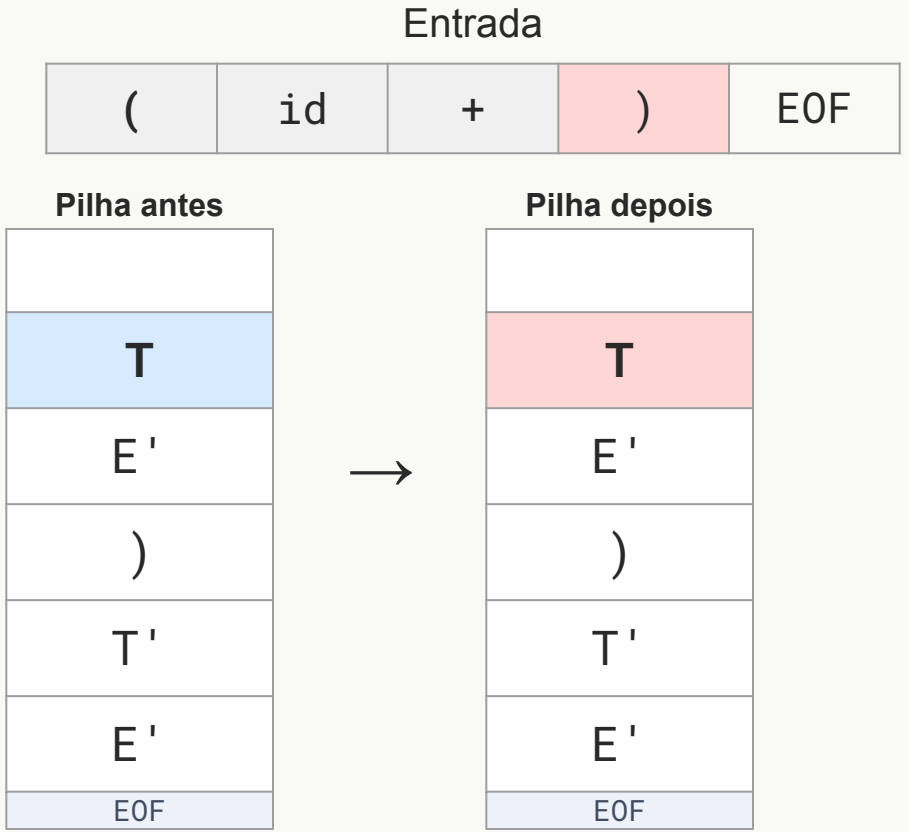
T
E'
)
T'
E'
EOF

Ação: consumir +

Uso da tabela M · passo 12/12

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		

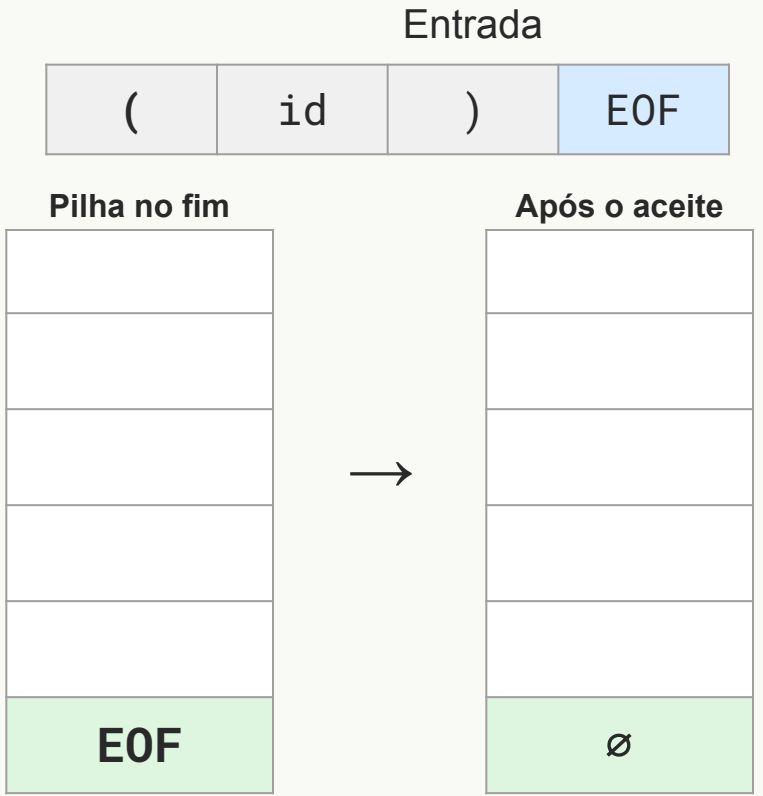


Erro: M[T,)] vazia — falta um termo após +.

Aceitação · exemplo válido (id)

NT	id	+	*
E	E→TE'		
E'		E'→+TE'	
T	T→FT'		
T'		T'→ε	T'→*FT'
F	F→id		

NT	(	)	EOF
E	E→TE'		
E'		E'→ε	E'→ε
T	T→FT'		
T'		T'→ε	T'→ε
F	F→(E)		



Aceitar: EOF na pilha e EOF na entrada.

Atividade

Atividade final: construa e use a tabela M

Símbolo inicial: E. Trate id como um único token.

```
E  → T E'
E' → + T E' | ε
T  → id P | ( E ) P
P  → * T | ε
```

Calcule FIRST e FOLLOW de E, E', T e P.

Construa a tabela M (id, +, *, (,), EOF).

Simule A e B: registre pilha, entrada e ação a cada passo; indique aceitação ou a célula onde ocorre erro.

A: id + id * id EOF
B: id * + id EOF